



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 6

STEPPER MOTOR CONTROL WITH FPGA

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 6

Stepper Motor Control with FPGA

Objectives

1. To design digital circuits in an FPGA to control a stepper motor.
2. To use Verilog as a design entry method in Altera Quartus II Software.
3. To simulate the circuit using Altera Quartus II Software.
4. To download the design onto an Altera DE2 board.

Equipment / Software Required

1. Altera Quartus II software
 - version 9/10/12 for Cyclone II
 - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Parallax 4-phase, 12 V, 75 Ω unipolar stepper motor
4. ULN2003 Darlington transistor array (motor driver)

Introduction

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied — it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor, 3.6° per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table 6.1 shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

Table 6.1: Normal step sequence for a four-coil unipolar stepper motor.

Coil	Step 1	Step 2	Step 3	Step 4	Step 1
1 (B)	1	1	0	0	1
2 ($\bar{B}$)	0	0	1	1	0
3 (A)	1	0	0	1	1
4 ($\bar{A}$)	0	1	1	0	0

Procedure

Activity 1 – Verilog stepper motor controller

Objective: write Verilog code to control the motion of a stepper motor.

1. Open Quartus II and start the New Project Wizard.
 - Working directory: fpga; project name: activity1; top-level entity: step.
 - Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N).
 - Accept the default EDA tool settings and finish.
2. Create a new Verilog HDL file and enter the code in Listing 6.1 (step, divider, and stepper modules).
3. Save as step and start compilation.
4. Use **Tools** → **Netlist Viewer** → **RTL Viewer** to generate the block overview shown in Figure 6.1.
5. Assign pin numbers as shown in Table 6.2 (refer to the DE2 user manual). For the phaseout outputs that interface with the stepper motor driver, use expansion header GPIO_0.
6. Download the program to the Altera DE2 board.
7. Discuss the result.

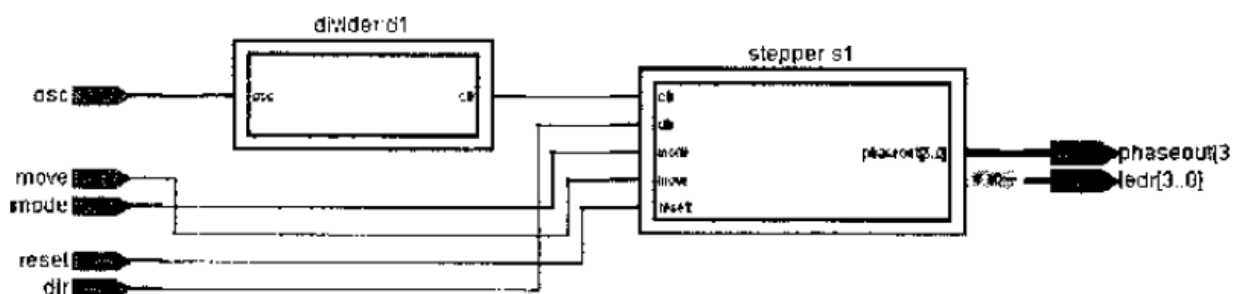


Figure 6.1: RTL Viewer overview of the step top-level module.

Table 6.2: Pin assignment for stepper motor control.

#	Node name	Direction	Pin location	#	Node name	Direction	Pin location
1	osc	Input		5	ledr[0]	Output	
	dir (SW0)	Input		6	phaseout[3]	Output	
	mode (SW1)	Input		7	phaseout[2]	Output	
	move (SW2)	Input		8	phaseout[1]	Output	
	reset (SW3)	Input		9	phaseout[0]	Output	
2	ledr[3]	Output					
3	ledr[2]	Output					
4	ledr[1]	Output					

step.v — top-level module

```

1 //=====
2 // Stepper Motor Controller
3 //=====
4 // TOP LEVEL MODULE
5 // reset (active low) brings the shaft to its reference position.
6 // mode selects step (1) or continuous (0) operation.
7 // dir selects clockwise (1) or anti-clockwise (0) motion.
8 // move shifts the shaft by one half-step in step mode.
9 module step(osc, reset, dir, mode, move, phaseout, ledr);
10     input  osc, reset, mode, dir, move;
11     output [3:0] phaseout, ledr;
12
13     wire clk;
14
15     divider d1 (
16         .osc (osc),
17         .clk (clk)
18     );
19
20     stepper s1 (
21         .clk      (clk),
22         .reset    (reset),
23         .dir      (dir),
24         .mode     (mode),
25         .move     (move),
26         .phaseout (phaseout)
27     );
28 endmodule

```

step.v – motor controller module (part 1 of 2)

```

1  // Module stepper is the motor controller.
2  module stepper(clk, reset, dir, mode, move, phaseout);
3      input  clk, reset, dir, mode, move;
4      output [3:0] phaseout;
5      reg [3:0] phaseout, position, ledr;
6
7      always @(posedge clk or negedge reset)
8      begin
9          if (reset == 0) begin
10             // motor resets to phase 1000 when reset is asserted
11             phaseout = 4'b1000;
12             ledr      = phaseout;
13         end
14         else begin
15             if (mode == 0) begin
16                 // Mode = 0: continuous motion.
17                 // The 8-state sequence below follows Table 1 in both
18                 // directions; only the first two states are shown here,
19                 // the remaining six states (0100, 0110, 0010, 0011,
20                 // 0001, 1001) follow the identical if(dir==0)/else
21                 // pattern, cycling back to 1000.
22                 case (phaseout)
23                     4'b1000: if (dir == 0) begin
24                         phaseout = 4'b1100; ledr = phaseout;
25                     end else begin
26                         phaseout = 4'b1001; ledr = phaseout;
27                     end
28                     4'b1100: if (dir == 0) begin
29                         phaseout = 4'b0100; ledr = phaseout;
30                     end else begin
31                         phaseout = 4'b1000; ledr = phaseout;
32                     end
33                     // ... 4'b0100, 4'b0110, 4'b0010, 4'b0011,
34                     //      4'b0001, 4'b1001 follow the same pattern ...
35                     default: $display("invalid input");
36                 endcase
37             end
38             else begin
39                 // Mode = 1: single-step motion, advances only when
40                 // move == 0. Uses the same 8-state sequence as above,
41                 // indexed by `position` instead of `phaseout`.
42                 if (move == 1'b0) begin
43                     case (position)
44                         // ... identical 8-state sequence as continuous
45                         //      mode, see Table 1 ...
46                         default: $display("invalid input");
47                     endcase
48                 end
49             end
50         end
51     end

```

```
51 end
```

step.v — motor controller module (part 2 of 2)

```
1  // Motor advances by one half-step on the falling edge of move.
2  always @(negedge move or negedge reset)
3  begin
4      if (reset == 0) begin
5          position = 4'b1000; // NOTE: corrected from the original
6                               // listing, which printed this value
7                               // as "4.101011" -- likely a
8                               // transcription error; 4'b1000
9                               // matches the reset state used
10                              // consistently elsewhere.
11          ledr = position;
12      end
13      else if (reset == 1) begin
14          position = phaseout;
15          ledr      = phaseout;
16      end
17  end
18 endmodule
```

step.v — clock divider module

```
1  // Module divider divides the incoming clock to produce a
2  // clock suitable for driving the motor controller.
3  module divider(osc, clk);
4      input  osc;
5      output clk;
6      reg    clk;
7      reg [15:0] count;
8
9      initial count = 16'b0000000000000000;
10
11     always @(posedge osc)
12     begin
13         count = count + 1;
14         clk    = count[15];
15     end
16 endmodule
```

Listing 6.1: step.v — top-level, motor-controller, and clock-divider modules.

Activity 2 — Variable-speed control with display feedback

Objective: extend the controller with selectable speed and direction/mode feedback.

1. Write Verilog code that allows selection of three different speeds for the stepper motor in continuous-motion mode.
2. Display the motor's direction of rotation on the seven-segment display, and the current mode (step / continuous) on the LCD display.

3. Print and submit the Verilog code and schematic diagram.

Appendix: Stepper Motor Datasheet Reference

Relevant specifications and wiring information for the Parallax #27964 stepper motor used in this experiment, extracted from the manufacturer datasheet (see References).



Figure 6.2: Parallax #27964 stepper motor.

Table 6.3: Technical specifications, Parallax #27964 stepper motor.

Parameter	Value
Type	4-phase, 12 V unipolar
Step angle	3.6° per step
Phase resistance	75 Ω
Current	150 mA
Phase inductance	39 mH
Detent torque	80 g·cm
Holding torque	600 g·cm
Mounting hole spacing (diagonal)	1.73 in
Mounting hole diameter	0.11 in
Shaft diameter	0.197 in
Shaft length	0.43 in
Motor diameter	1.66 in
Motor height	1.35 in
Weight	0.55 lb

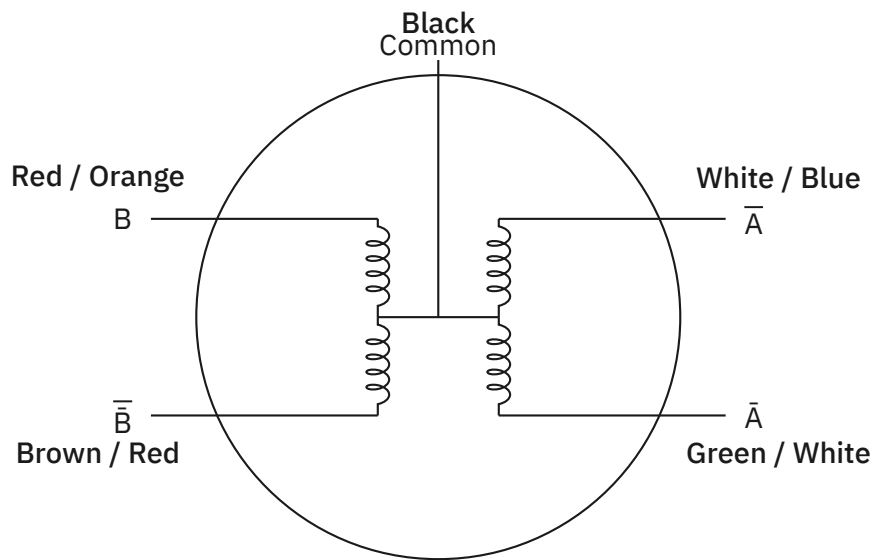


Figure 6.3: Coil winding and lead-wire colours, Parallax #27964 stepper motor.

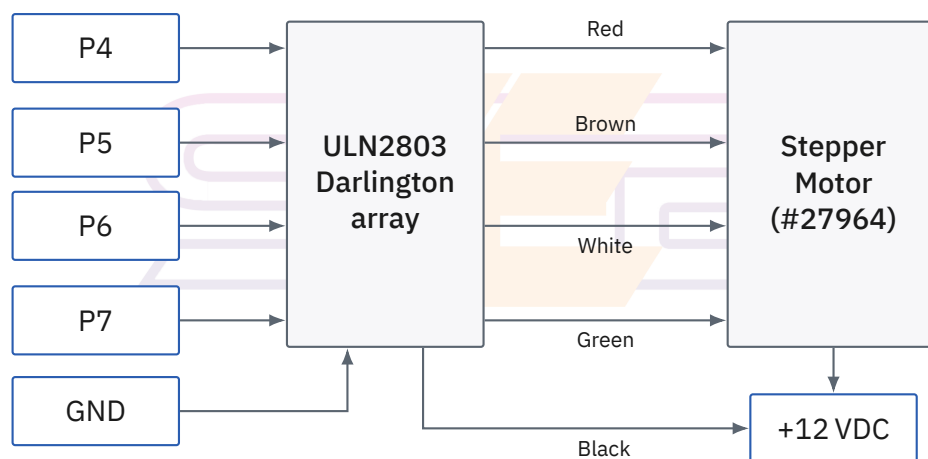


Figure 6.4: Reference driver-wiring diagram.

Note:

1. The P4–P7 labelling above follows the datasheet’s original microcontroller example; in this experiment, the corresponding driver inputs are instead the phaseout [3:0] outputs of the stepper module, assigned to DE2 GPIO pins as listed in Table 6.2.
2. The GND are to be common ground with the DE2 board, ULN2803 and the power source.
3. The +12 VDC is to be supplied from an external power source, not the DE2 board.

References

1. Altera DE2 User Manual.
2. *Introduction to Quartus II Software – Verilog, VHDL, Schematic*, Altera University Program.

3. *Introduction to Verilog*, Carleton University course notes.
4. *Stepper Motor Controller Using MAX II CPLDs*, Altera Application Note AN-488.
5. Parallax Inc., *4-Phase 12-Volt 75 Ω Unipolar Stepper Motor (#27964)*, technical datasheet.

